

Base de données avancée
Rapport - Contrôle continu n° 2

Eldis YMERAJ - 22408331
Redwan OMARI - 22406817
Master 1 - Intelligence artificielle et facteurs humains

24 octobre 2024



**RÉPUBLIQUE
FRANÇAISE**

*Liberté
Égalité
Fraternité*



**UNIVERSITÉ
CAEN
NORMANDIE**

Table des matières

1	Création d'un entrepôt de données	3
1.1	Modélisation en étoile	3
1.2	Schéma relationnel et les scripts SQL	3
2	Alimentation de l'entrepôt	6
2.1	Pipeline d'alimentation des dimensions Dates et Géographie	6
2.1.1	Lecture des données	6
2.1.2	Alimentation des tables	6
2.1.3	Filtrage des erreurs géographiques	6
2.1.4	Écriture des données valides	6
2.2	Pipeline d'alimentation des dimensions Prestations et Clients	7
2.2.1	Lecture des données	7
2.2.2	Transformation et tri des données	7
2.2.3	Gestion des adresses clients	7
2.2.4	Alimentation des tables	7
2.2.5	Utilisation du numéro de client	7
2.3	Pipeline pour l'alimentation de la table de faits	8
2.3.1	Lecture des données :	8
2.3.2	Transformation des données :	8
2.3.3	Alimentation de la table des faits :	9

Table des figures

1	Modélisation en étoile des données.	3
2	Schéma détaillé des tables de faits et des dimensions.	4
3	Pipeline 1.	6
4	pipeline 2.	8
5	pipeline 3.	9

Introduction

La première partie de notre travail a consisté à modéliser les données de vente et de clients sous la forme d'une étoile, en prenant en compte divers axes d'analyse tels que le chiffre d'affaires, la durée des interventions, et la répartition géographique des prestations. Nous avons ensuite implémenté cette structure dans une base de données relationnelle à l'aide de scripts SQL.

Dans la deuxième partie, nous avons créé des pipelines dans Apache Hop pour extraire les données de sources opérationnelles, les transformer en fonction des besoins de l'entrepôt, et les charger dans les différentes tables dimensionnelles et factuelles du datawarehouse.

1 Création d'un entrepôt de données

1.1 Modélisation en étoile

Dans notre projet, la table centrale *Vente* contient les faits mesurables tels que le prix des prestations, la durée des interventions, et les informations géographiques. Cette table est reliée à quatre dimensions principales : *Date*, *Localisation*, *Client*, et *Prestation*, chacune jouant un rôle essentiel dans les analyses.

- **La dimension Date** permet d'effectuer des analyses temporelles sur différents niveaux de granularité (année, mois, jour, heure, minute).
- **La dimension Localisation** regroupe les informations géographiques des ventes et des clients, telles que la ville, le code postal et la région.
- **La dimension Client** fournit les informations relatives aux clients, telles que leur nom, adresse, et autres détails personnels.
- **La dimension Prestation** décrit les services fournis, leur type, ainsi que la catégorie de prestation.

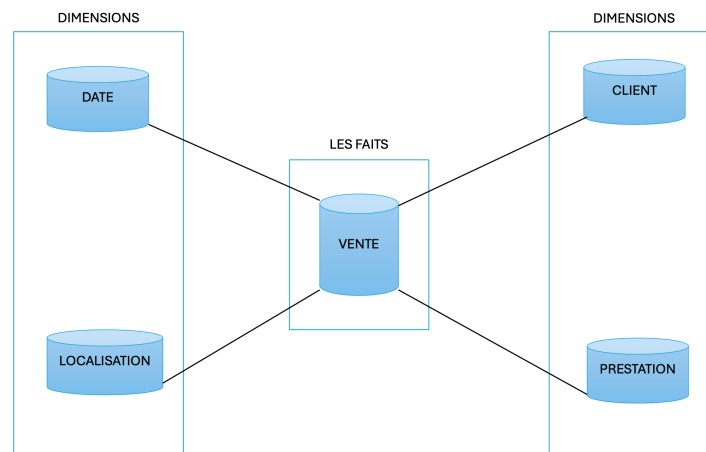


FIGURE 1 – Modélisation en étoile des données.

1.2 Schéma relationnel et les scripts SQL

Notre modélisation veille à insérer les éléments nécessaires pour que les données soient résilientes aux changements (comme les adresses des clients) et pour permettre la traçabilité des données. Par exemple, dans la dimension *Client*, nous avons introduit trois clés pour chaque client : une clé primaire (*id_technique*), une clé de business (*cle_client*), et une clé de substitution (*num_client*).



FIGURE 2 – Schéma détaillé des tables de faits et des dimensions.

- La clé primaire `id_technique` identifie de manière unique chaque enregistrement dans la table, ce qui garantit l'intégrité des données.
- La clé de business `cle_client` et la clé de substitution `num_client` permettent de gérer les cas où un même client peut avoir différentes versions dans le temps ou être représenté différemment dans d'autres systèmes d'information.

Ce choix de modélisation permet ainsi de suivre l'évolution des clients tout en maintenant une traçabilité complète des données, notamment lors des changements d'adresses ou d'autres informations. Le modèle relationnel utilise également les chaînes datetime comme clés pour les dates, garantissant l'unicité et la traçabilité des données temporelles.

Les scripts SQL nécessaires pour créer les tables de ce schéma relationnel sont disponibles dans le fichier `datawarehouse.txt` situé dans le répertoire `data_out`. Ces scripts créent les tables de l'entrepôt de données, qui seront alimentées ensuite via Apache Hop. Ci-dessous, vous trouverez le contenu du fichier `datawarehouse.txt` :

```

1  —La table des VENTES
2  DROP TABLE IF EXISTS Vente;
3  CREATE TABLE Vente (
4      vente_id          INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
5      id_technique       INTEGER NOT NULL REFERENCES Client(id_technique),
6      cle_prestation     INTEGER NOT NULL REFERENCES Prestation(cle_prestation),
7      id_localisation    INTEGER NOT NULL REFERENCES Localisation(id_localisation),
8      cle_date_debut     INTEGER NOT NULL REFERENCES Dates(date_complete),
9      cle_date_fin       INTEGER NOT NULL REFERENCES Dates(date_complete),
10     prix               FLOAT NOT NULL,
11     adresse            TEXT NOT NULL,
12     duration           INTEGER NOT NULL,
13     code_postal        TEXT NOT NULL,
14     ville              TEXT NOT NULL,

```

```

15     date_insertion  TEXT    NOT NULL
16 );
17 — La table Client
18 DROP TABLE IF EXISTS Client;
19 CREATE TABLE Client (
20     id_technique    INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
21     cle_client      INTEGER NOT NULL,
22     num_client      TEXT NOT NULL,
23     nom             TEXT NOT NULL,
24     prenom          TEXT NOT NULL,
25     adresse         TEXT NOT NULL,
26     code_postal     TEXT NOT NULL,
27     ville           TEXT NOT NULL,
28     department_code TEXT NOT NULL,
29     department_nom  TEXT NOT NULL,
30     region_code     TEXT NOT NULL,
31     region_nom      TEXT NOT NULL,
32     date_debut      NULL,
33     date_fin        NULL
34 );
35 — La table Prestation
36 DROP TABLE IF EXISTS Prestation;
37 CREATE TABLE Prestation (
38     cle_prestation  INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
39     code_prestation TEXT NOT NULL,
40     nom_prestation  TEXT NOT NULL,
41     categorie_prestation TEXT NOT NULL
42 );
43 — La table Dates
44 DROP TABLE IF EXISTS Dates;
45 CREATE TABLE Dates (
46     date_complete TEXT PRIMARY KEY NOT NULL,
47     annee          INTEGER NOT NULL,
48     trimestre     INTEGER NOT NULL,
49     mois           INTEGER NOT NULL,
50     nom_mois       TEXT NOT NULL,
51     jour_mois      INTEGER NOT NULL,
52     nom_jour       TEXT NOT NULL,
53     heure          INTEGER NOT NULL,
54     minute         INTEGER NOT NULL
55 );
56 — La table Localisation
57 DROP TABLE IF EXISTS Localisation;
58 CREATE TABLE Localisation (
59     id_localisation INTEGER PRIMARY KEY NOT NULL,
60     ville           TEXT NOT NULL,
61     insee           TEXT NOT NULL,
62     code_postal     TEXT NOT NULL,
63     department_code INTEGER NOT NULL,
64     department_nom  TEXT NOT NULL,
65     region_code     INTEGER NOT NULL,
66     region_nom      TEXT NOT NULL
67 );

```

Listing 1 – Scripts SQL pour la création des tables

2 Alimentation de l'entrepôt

2.1 Pipeline d'alimentation des dimensions Dates et Géographie

Pour l'exercice 4, nous avons conçu un pipeline dans Apache Hop qui alimente deux dimensions de notre entrepôt de données : la dimension des **dates** et celle des **lieux de réalisation des prestations**.

2.1.1 Lecture des données

Nous avons commencé par lire les fichiers `dates.csv` et `geography.csv`, contenant respectivement les données sur les dates (limitées à la période 2019-2021) et les lieux (avec les codes INSEE et les codes postaux).

2.1.2 Alimentation des tables

Pour la dimension des dates, après lecture des données, le pipeline alimente directement la table correspondante dans le datawarehouse sans transformations particulières, vu que les données sont conformes.

2.1.3 Filtrage des erreurs géographiques

Pour la dimension géographique, nous avons appliqué un filtre afin d'exclure les lignes contenant des codes INSEE ou postaux vides. Les lignes invalides sont envoyées dans un fichier d'erreurs via un nœud de sortie dédié. Cela permet d'assurer que seules les données valides sont insérées dans le datawarehouse.

2.1.4 Écriture des données valides

Les données valides sont ensuite insérées dans la table de sortie appropriée, correspondant aux lieux de prestation.

Ce pipeline est donc conçu pour traiter les données des deux dimensions tout en s'assurant que les données incohérentes ou incomplètes sont isolées pour une gestion ultérieure.

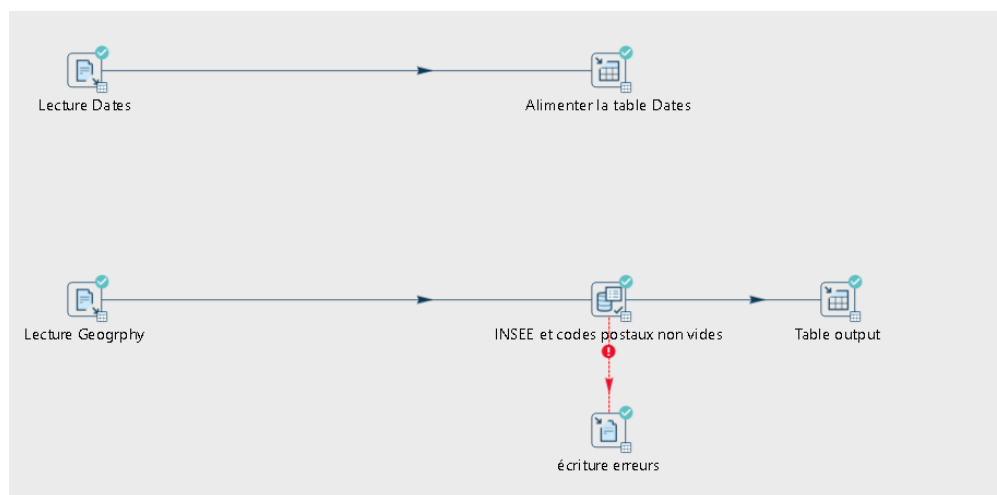


FIGURE 3 – Pipeline 1.

2.2 Pipeline d'alimentation des dimensions Prestations et Clients

Pour l'exercice 5, nous avons créé un pipeline dans Apache Hop qui alimente les autres dimensions du datawarehouse, en particulier la dimension des **prestations** et celle des **clients**, en appliquant les transformations nécessaires.

2.2.1 Lecture des données

Nous avons d'abord lu les fichiers contenant les informations des prestations ainsi que les données de géographie et clients depuis les tables existantes. Les données de géographie ont été triées avant d'être utilisées dans une jointure avec la table des clients.

2.2.2 Transformation et tri des données

Le pipeline trie les données des clients et les données géographiques pour les préparer à une **jointure**(on tri par code-postal et ville). La jointure permet de fusionner les informations clients avec les données géographiques correspondantes.

2.2.3 Gestion des adresses clients

Les adresses des clients sont considérées comme valides depuis une date inconnue et jusqu'à nouvel ordre, conformément aux indications fournies. Par conséquent, les dates de validité dans la table des clients sont laissées à NULL.

2.2.4 Alimentation des tables

Après avoir joint et trié les données par numéro de client, les informations sont insérées dans la table des clients du datawarehouse. Les prestations sont également lues et insérées directement dans la table correspondante après lecture des données.

2.2.5 Utilisation du numéro de client

Le numéro de client est utilisé à la fois comme **business key** et comme **surrogate key**, puisque c'est la clé primaire de la table des clients. Cette simplification permet une insertion plus directe des données dans le datawarehouse.

Ce pipeline gère efficacement l'alimentation des tables de prestations et de clients tout en respectant les contraintes de structure du datawarehouse, en particulier pour la gestion des adresses clients et des clés.

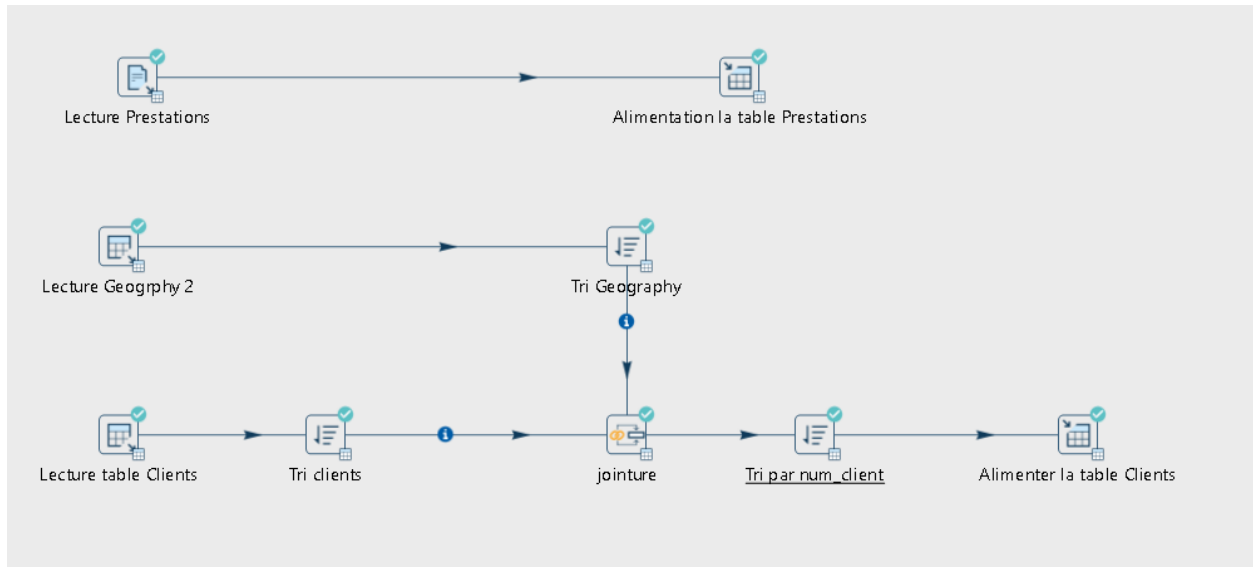


FIGURE 4 – pipeline 2.

2.3 Pipeline pour l'alimentation de la table de faits

Pour l'exercice 6, nous avons conçu un pipeline dans Apache Hop qui permet d'alimenter la **table de faits** en prenant les données de la table des **ventes**, tout en appliquant les transformations nécessaires pour enrichir et structurer les informations avant leur insertion dans le datawarehouse.

2.3.1 Lecture des données :

- Nous avons commencé par lire la **table des ventes** ainsi que les **dimensions** nécessaires telles que les **dates**, **clients**, **localisation** et **prestations** depuis le datawarehouse.
- Les données lues sont ensuite triées pour préparer les futures jointures.

2.3.2 Transformation des données :

1. Jointure des dates :

- Après avoir trié les dates de début et de fin, j'ai utilisé une **jointure** pour combiner les heures de début et de fin avec les dates complètes des événements. Les colonnes de date ont été renommées pour faciliter les manipulations.
- Ensuite, un calcul a été effectué pour **déterminer la durée** des prestations en heures, en utilisant l'opération « Date A - Date B (minutes) ».

2. Jointure avec les clients et localisation :

- Les données clients sont triées par numéro de client, puis jointes aux ventes pour enrichir les informations sur chaque transaction.
- De la même manière, une **jointure** avec les informations de localisation a été réalisée, en triant par **code postal** et **ville**.

3. Jointure des prestations :

- Enfin, les prestations sont jointes aux ventes pour associer chaque vente avec la prestation correspondante.

2.3.3 Alimentation de la table des faits :

Après toutes ces jointures et transformations, les données enrichies et validées sont insérées dans la **table des faits** du datawarehouse.

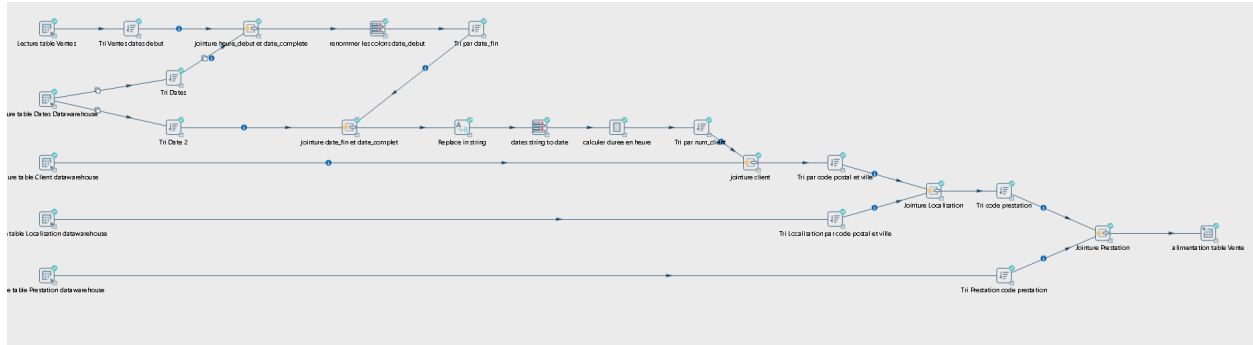


FIGURE 5 – pipeline 3.

Conclusion

Ce pipeline prend en charge la fusion des informations critiques à partir de différentes dimensions, calcule la durée des prestations et permet d'enrichir la table des faits pour une analyse complète et précise des données de ventes.